Drawing from control theory again, we can say that manageability measures how easily and efficiently an external input can change the state or the output of a system. In less mathematical terms, it's the ability to modify an application's behavior without needing to change its code. This is not to be confused with *maintainability*, which measures how easily and efficiently you can change a system from the inside by changing its code. Figure 1.6 shows how manageability works.

One aspect of manageability is deploying and updating applications while keeping the overall system up and running. Another element is configuration, which I'll address in depth throughout the book. We want to make cloud native applications configurable so we can modify their behavior without changing their code and building a new release. It's common to make configurable settings like data source URLs, service credentials, and certificates. For example, depending on the environment, you may use different data sources: one for development, one for testing, and one for production. Other types of configuration could be feature flags, which determine whether specific features should be enabled at runtime. I'll show you different strategies for configuring applications throughout the book, including using Spring Cloud Config Server, Kubernetes ConfigMaps and Secrets, and Kustomize.

Manageability is not only about particular changes themselves, but also about how easily and efficiently you can apply those changes. Cloud native systems are complex, so it's essential to design applications that can adapt to changing requirements regarding functionality, the environment, and security. Due to this complexity, we should aim at managing as much as possible through automation, which leads us to the last of the Three Ps of Cloud Native: practices.